

70-467 Machine Learning for Business Analytics

Evaluating model accuracy and cross-validation

Professor: Andrés Castaño Zuluaga

7 September 2026 · CMU-Q

Discussion 2

Job interviews are becoming AI tests.

- 1 McKinsey says they do not hire on how you prompt. They look at what you do with the output. Why would a firm hire on that?
- 2 The tools will change. What from this course would still show up in that interview?

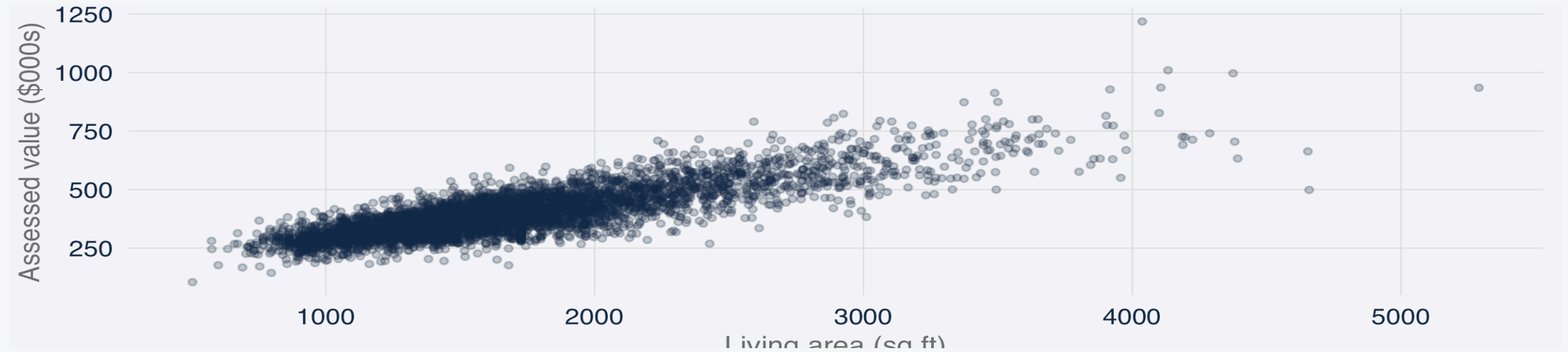
Both questions are already on Canvas. 100–200 words was the post.

Predicting house values

- You have been given data for 5,802 single-family, owner-occupied houses in West Roxbury, a neighborhood in southwest Boston, from the 2014 city assessing roll. The City of Boston Assessing Department publishes parcel-level assessed values and physical characteristics for property tax purposes.
- **Outcome Y :** `TOTAL.VALUE` → assessed value, in thousands of dollars. Mean 393, SD 99.
- **We start with:** `LIVING.AREA` → living area, in square feet.
- Decision this hour: we need to predict prices for a **new** house that's not in the file, which error do we believe the one on houses we already fitted, or the one on houses we held out?

Descriptive relationship: Assessed value against living area

- $n = 5,802$. What do you see in the graph?



Variables in the dataset

Variable	Definition
TOTAL.VALUE	Assessed value (\$000s) — the <i>Y</i>
LIVING.AREA	Living area (sq ft)
TAX	Property tax bill (dollars)
LOT.SQFT	Lot size (sq ft)
YR.BUILT	Year built
GROSS.AREA	Gross area (sq ft)
REMODEL	None / Old / Recent
ALSO IN THE FILE	
FLOORS, ROOMS, BEDROOMS, FULL.BATH, HALF.BATH, KITCHEN, FIREPLACE	Counts

Outline of today

- 1 Measuring the quality of fit (MSE, and how it sits next to last week's RSE)
- 2 Training vs test samples
- 3 Bias, variance, over/under fitting
- 4 Validation set, cross-validation, and how we pick complexity

Important concepts when evaluating model accuracy

Measuring quality of fit I: MSE

Last week we judged the line with RSE in thousands of dollars. If a house is \$20k off the line, how should that miss enter a single score we can later compute on houses we did **not** fit?

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2.$$

- y_i is this house's assessed value.
- $\hat{f}(x_i)$ is the fitted value.
- Square so residuals do not cancel.
- Divide by n so it is an average squared residual.

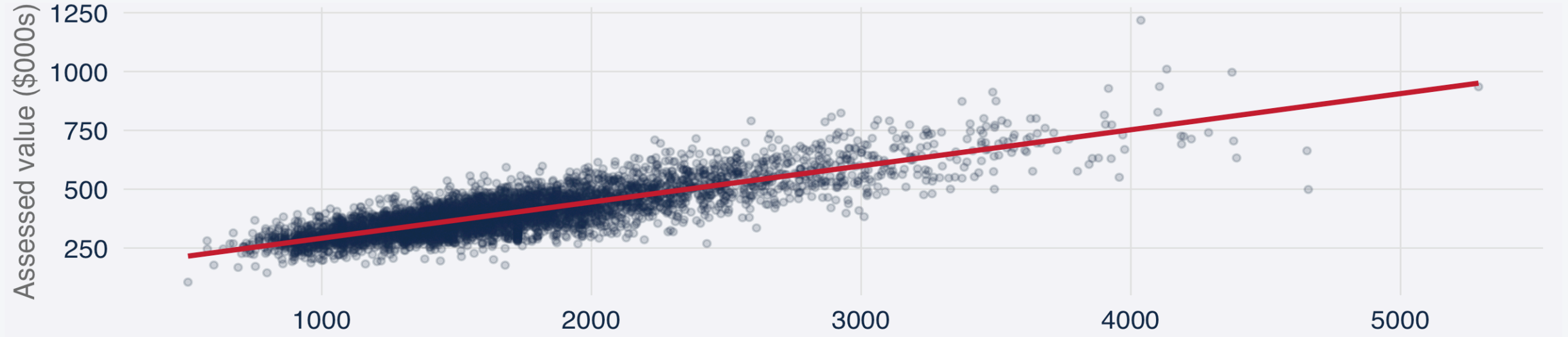
Different ways to approach accuracy: MSE, RMSE, and RSE

- Same residuals $e_i = y_i - \hat{y}_i$. Three ways to scale them:

$$\text{RMSE} = \sqrt{\text{MSE}}, \quad \text{RSE} = \sqrt{\frac{1}{n - p - 1} \sum_{i=1}^n e_i^2}.$$

- RMSE and MSE use n .
- RSE uses residual degrees of freedom.
- RSE is in the units of Y , like last week.
- MSE is the number we will split into train vs test.

Fitted line: assessed value on living area



Living area (sq ft). The residual is the vertical miss.

MSE in R

```
1 m <- lm(TOTAL.VALUE ~ LIVING.AREA, data = houses)
2 mse <- mean(resid(m)^2)
3 rmse <- sqrt(mse)
4 rse <- sigma(m)
```

- $MSE = 2,942.8$. $RMSE = 54.2$. $RSE = 54.3$.
- A typical miss is about 54.3 thousand dollars.
- That is still an **in-sample** number: every house was used to fit the line. What about an out sample number?

Measuring quality of fit II: training and test data

Headquarters fitted on every house and reported a small MSE. A house not in the file arrives tomorrow. Is that MSE a useful number for the new house?

- **Training** rows: used to choose $\hat{f}$.
- **Test** rows: never used to fit; used only to get a prediction.
- Is the MSE from testing data still useful predict observations in the testing data?

Training MSE and test MSE

$$\text{MSE}_{\text{train}} = \frac{1}{n_{\text{train}}} \sum_{i \in \text{train}} (y_i - \hat{f}(x_i))^2, \quad \text{MSE}_{\text{test}} = \frac{1}{n_{\text{test}}} \sum_{j \in \text{test}} (y_j - \hat{f}(x_j))^2$$

- $\hat{f}$ is fit on **train only**.
- MSE_{test} is the number we believe for a house we have not seen.

A 50/50 split

```
1 set.seed(467)
2 idx  <- sample(nrow(houses), floor(0.5 * nrow(houses)))
3 train <- houses[idx, ]
4 test  <- houses[-idx, ]
```

Train

2901 houses
fit model here

Test

2901 houses
get predictions (scores) here

Problem: How do you select a model without a test set?

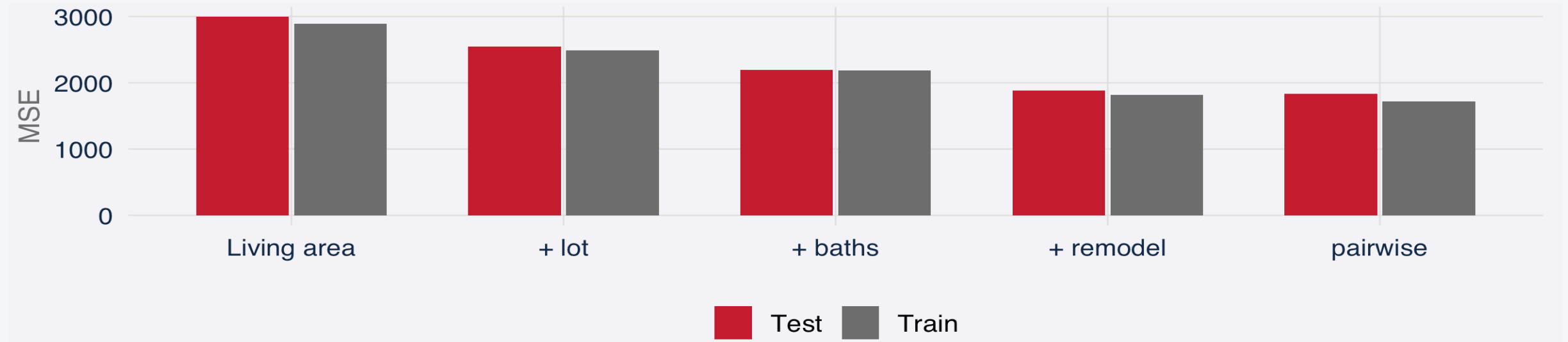
How do you select the best model if you do **not** have a test set?

Training MSE and test MSE can tell different stories

If besides living area, we add lot size, baths, remodel, and then a pile of interactions, will the residuals (MSE) on houses we did not fit go down?

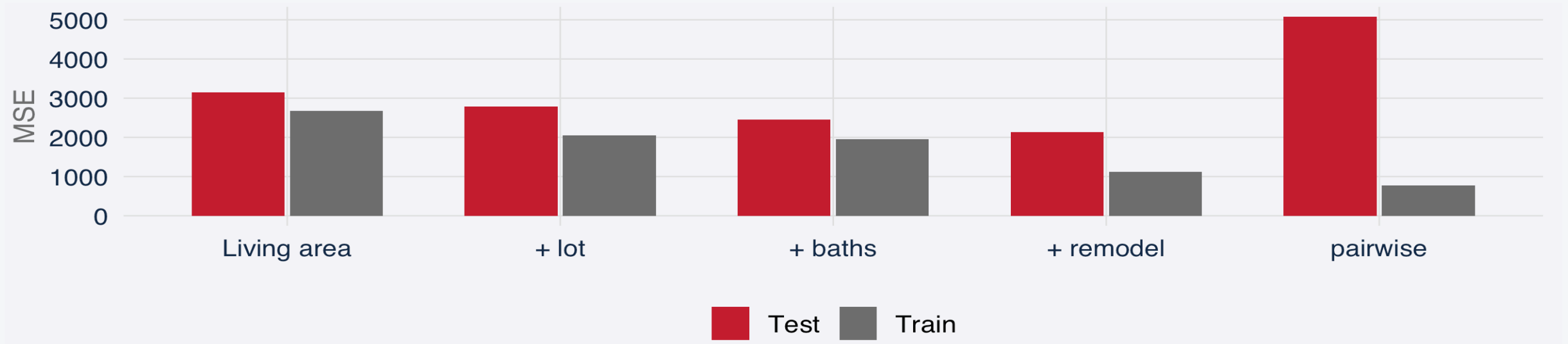
- Living area: `TOTAL.VALUE ~ LIVING.AREA`
- ■ lot: `+ LOT.SQFT`
- ■ baths: `+ FULL.BATH + HALF.BATH + FIREPLACE`
- ■ remodel: `+ REMODEL + FLOORS + GROSS.AREA`
- pairwise: the + remodel columns, plus every two-way among the numeric ones
- what do you think?

Result: MSE as we add predictors (50/50 split)



50/50 split, seed 467. Test MSE falls from 2,997 (living area) through + remodel (1,881) to nested pairwise (1,833). Extra columns still help at this n .

Training with 80 houses: MSE as we add predictors

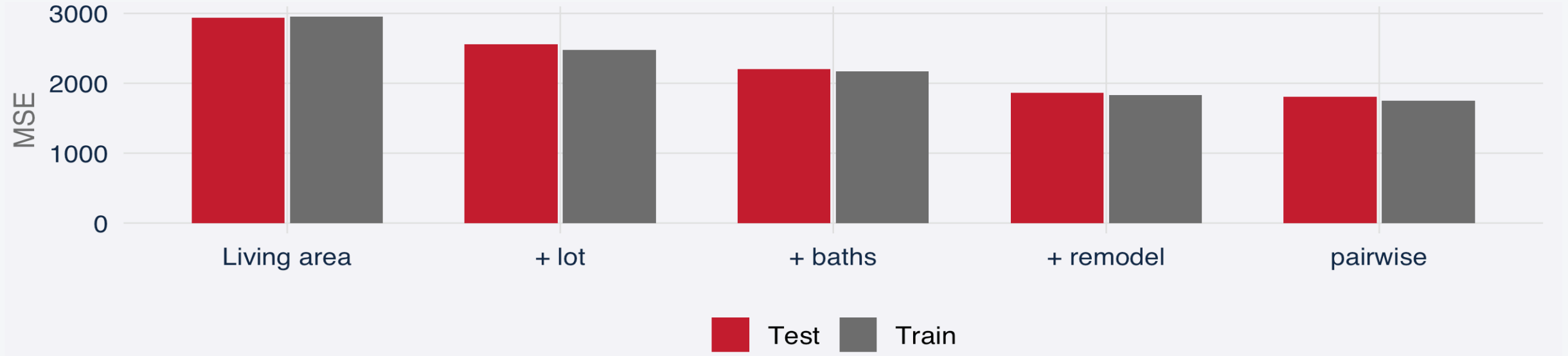


- What do you see?

Same columns, only 80 training houses. Pairwise hugs those 80 (train MSE 775) and misses the rest (test 5,079).

A second split

If I shuffle again, does test MSE still fall as we add columns?



Seed 468. Test MSE again falls through pairwise (1,809 vs + remodel 1,861). The overfit is not this split.

The bias–variance trade-off: equation and intuition

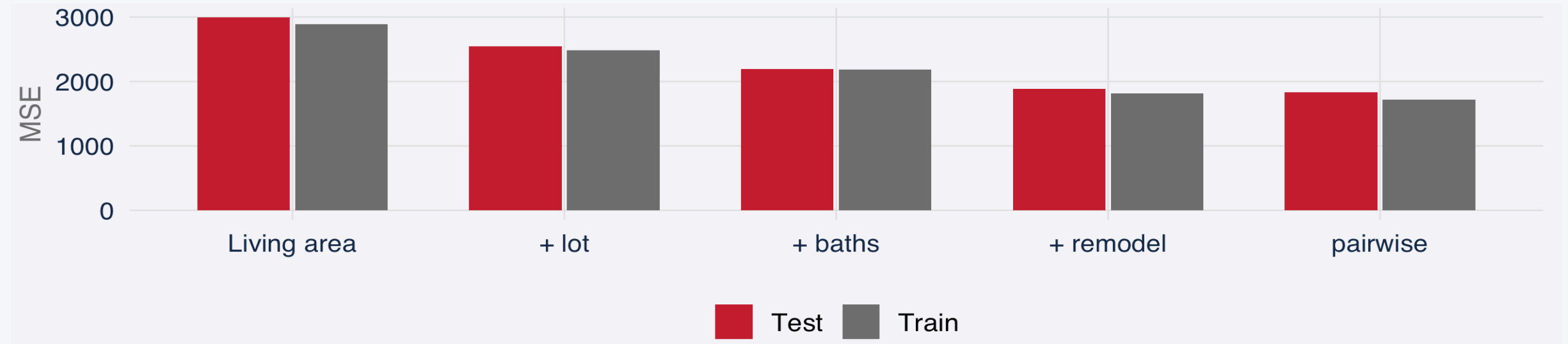
- Extra columns cut the test MSE on 5,802 houses. However,
- When the training test was 80 houses a 31-coefficient model blew up on 80 houses.

$$E[(y_0 - \hat{f}(x_0))^2] = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\varepsilon).$$

- **Variance** of $\hat{f}(x_0)$:
 - How much would the prediction at this house change if we change the training sample?
- **Bias** of $\hat{f}(x_0)$: $E[\hat{f}(x_0)] - f(x_0)$.
 - Prediction you would get if you could train the same model on every possible sample of houses and then average those predictions. Answer the question: How far is the average prediction from the true expected price?
 - Bias could be high no matter how many observations you have if the model only includes a limited set of features (say living area in sq meters)
- **Irreducible error** $\text{Var}(\varepsilon)$: miss even with the true f . A floor.

This file on the bias-variance picture

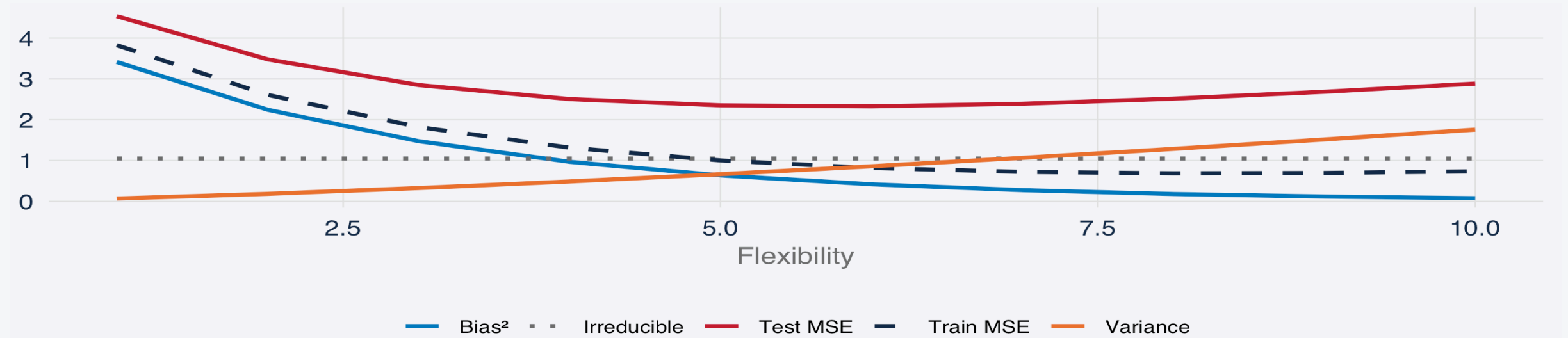
On a picture of Var / Bias² / test MSE vs flexibility, where is a model too simple? Too flexible? Where is **this** file?



- Intuition: Making the model more flexible (more features, more interactions) shrinks the Bias² because the average prediction gets closer to $f(x)$. However, the same flexibility inflates the Variance because the fit becomes more sensitive to whichever houses happened to be in the training set.

This file, 50/50. Living area is too simple. Nested pairwise still helps. The too-flexible picture is 80 houses, not this split. We did not compute Var / Bias² from one split.

Bias², variance, and test MSE



Bias² falls, variance rises, test MSE is U-shaped.

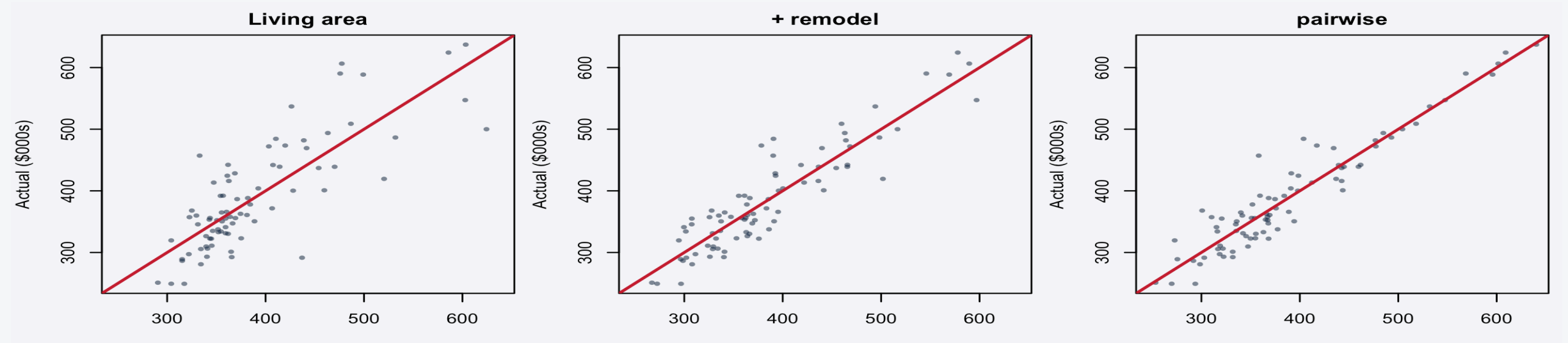
Overfitting vs underfitting

Actual vs fitted on that small training file.

```
1 set.seed(467)
2 tiny <- houses[sample(nrow(houses), 80), ]
3 m_ok <- lm(TOTAL.VALUE ~ LIVING.AREA + LOT.SQFT + FULL.BATH + HALF.BATH +
4           FIREPLACE + REMODEL + FLOORS + GROSS.AREA, data = tiny)
```

Three fits on 80 houses

Which of these three would you trust for a house **not** in this picture?



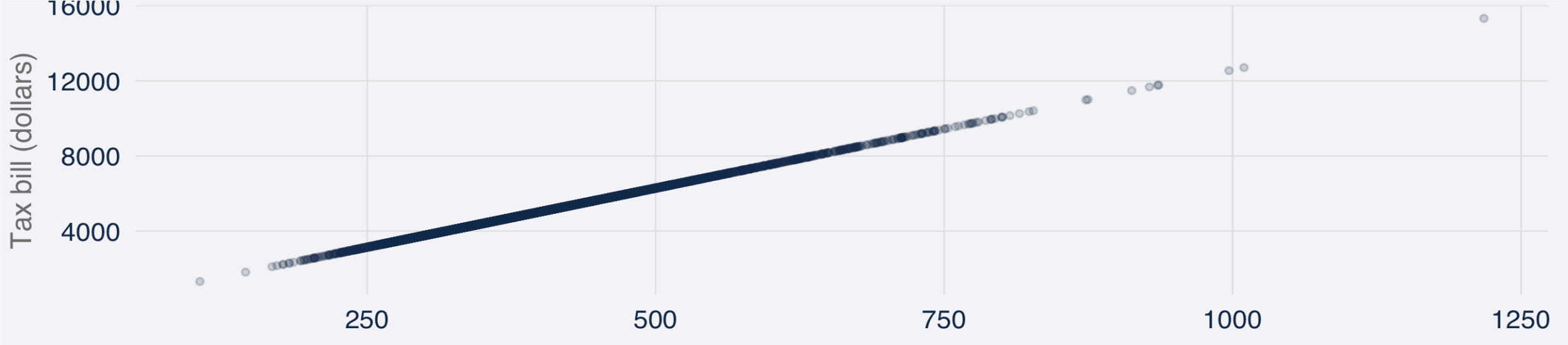
Fitted vs actual on the 80. Living area: high bias. Pairwise hugs the 80 (train MSE 775); on the other houses, test MSE is 5,079.

Leakage: information you shouldn't have

Would you have the tax bill for a house you have **not assessed yet**?

- A column that is the answer in disguise
- Peeking at the test rows to pick the model
- A split that uses information from after the decision
- This file: `TAX` is the tax **bill**.
- A leakage is feature that carries information you would not have at the moment you must issue a prediction
- **Why using this could be problematic for a prediction model?**

TAX versus assessed value



Assessed value (\$000s). About \$12.58 per \$1,000 of assessed value.

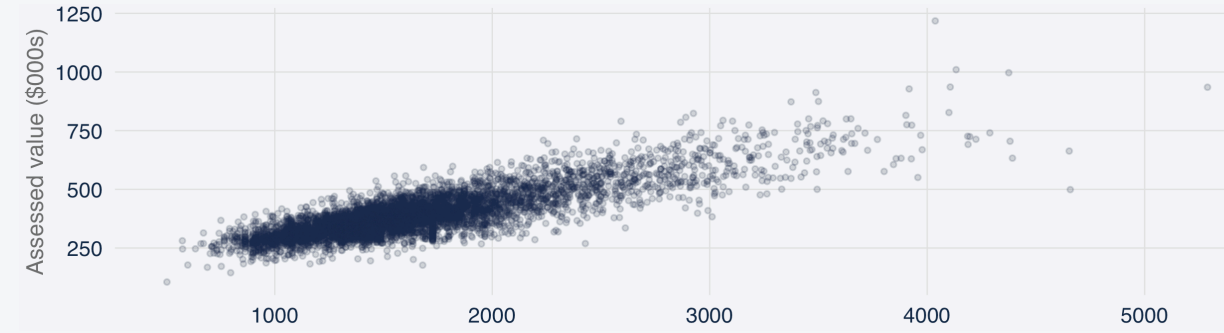
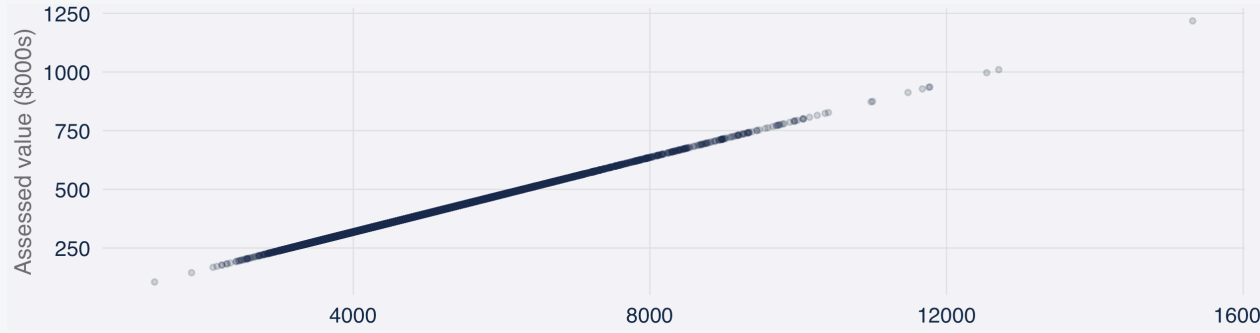
TAX is a worked leak

Overfit looked great on the houses we fitted and worse on the ones we held out. If TAX is a leak rather than a flexible line, what should test MSE look like — worse than train, or also tiny?

```
1 m_tax  <- lm(TOTAL.VALUE ~ TAX, data = train)
2 m_area <- lm(TOTAL.VALUE ~ LIVING.AREA, data = train)
3 c(tax_train  = mean((train$TOTAL.VALUE - fitted(m_tax))^2),
4   tax_test   = mean((test$TOTAL.VALUE - predict(m_tax, test))^2),
5   area_train = mean((train$TOTAL.VALUE - fitted(m_area))^2),
6   area_test  = mean((test$TOTAL.VALUE - predict(m_area, test))^2))
```

	Train MSE	Test MSE
TAX	0.0005	0.0005
Living area	2,892.3	2,997.1

TAX versus living area



Left: TAX (dollars). Right: living area (sq ft). Leak looks good on **test** too. Do not put TAX in any later model.

Cross-validation

Motivation for validation

The error on houses we fitted is the wrong number to report, and one 50/50 split can jump. What do we do if we **cannot** hold out a large, once-and-for-all test set?

We reuse the training rows to mimic a test set (resampling).

We can do this for different purposes:

- **Assessment:** how well will this predict a new house? (test error)
- **Selection:** which model (proper level of flexibility)?

The validation-set approach

We are going to reuse the training rows **once**. In what order do we split, fit, and score so the number is not the training error?

$$\text{MSE}_{\text{val}} = \frac{1}{n_{\text{val}}} \sum_{j \in \text{val}} (y_j - \hat{f}_{\text{train}}(x_j))^2.$$

Shuffle, split, fit, score

1. Shuffle

Random split.

2. Split

Some observations to train set,
the rest to validation set.

3. Fit on train

Choose the fitted model using
only the training rows.

4. Predict on validation set

t.

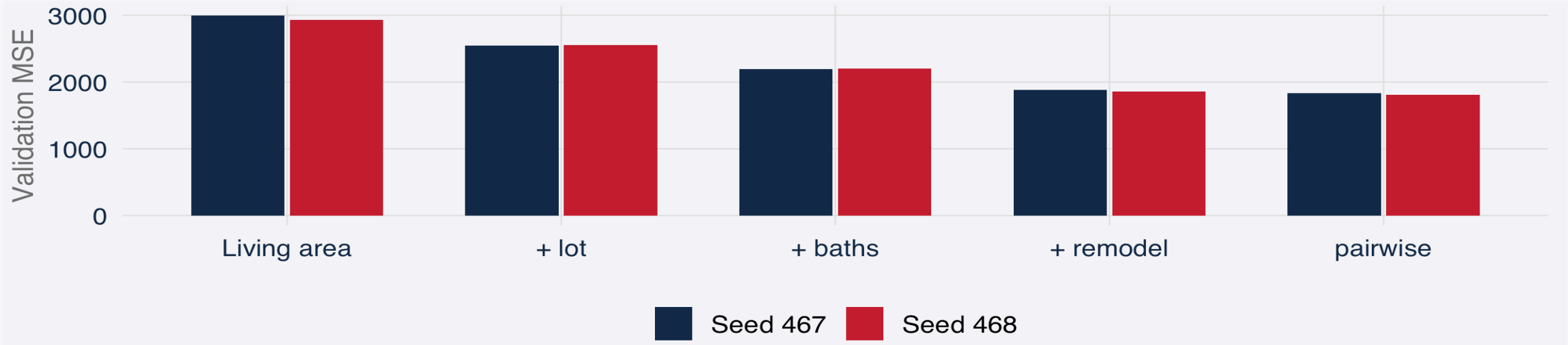
Validation set on our data (and why we need more)

We reuse the rows **once**. If I shuffle again, do I pick the same columns?

```
1 val_mse <- function(form, tr, va) {  
2   m <- lm(form, data = tr)  
3   mean((va$TOTAL.VALUE - predict(m, va))^2)  
4 }
```

- First bars: the seed-467 split we already have. Second: `set.seed(468)` only.

This file: two validation curves



Both splits still fall through pairwise. Extra columns still help at $n = 5,802$. The height moves a little with the shuffle.

Limitations of the validation set

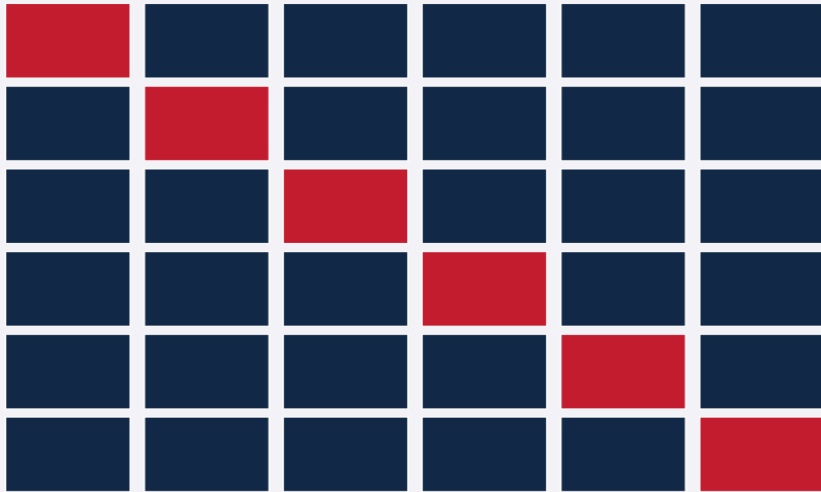
- MSE in the validation set is **noisy** (depends on who landed in the training and validation set).
- Since the model is trained on **fewer** data points, so validation MSE tends to **overestimate** test error.
 - Statistical methods tend to perform worse when trained in fewer observations.

Solution: Two kinds of cross-validation

On 80 houses, training MSE was decent, but a disaster on test observations.

How can we reuse the training observations **many** times so the error is not just a single observation?

Leave-one-out CV



Each house is held out once. n fits.

K-fold CV



Each block is the test set once. $k = 5$ or 10 .

The two CV averages

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n \text{MSE}_i, \quad CV_{(k)} = \frac{1}{k} \sum_{\ell=1}^k \text{MSE}_{\ell}.$$

- LOOCV is k-fold with $k = n$. The number we report is the **average** of the test MSEs.
- LOOCV pros and cons:
 - (+) Less bias as each training set contains most of the info
 - (-) Computationally expensive (model has to be fit n times)
- K-fold CV pros and cons
 - (+) Computationally more efficient than LOOCV
 - (+) Often gives more accurate estimates of the test error than LOOCV (more balance between bias and variance)

LOOCV on our data, including the OLS shortcut

Leave-one-out means we refit once per house. We have 5,802 houses. Do we actually run 5,802 regressions in this hour?

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i^{(-i)})^2 = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - h_{ii}} \right)^2$$

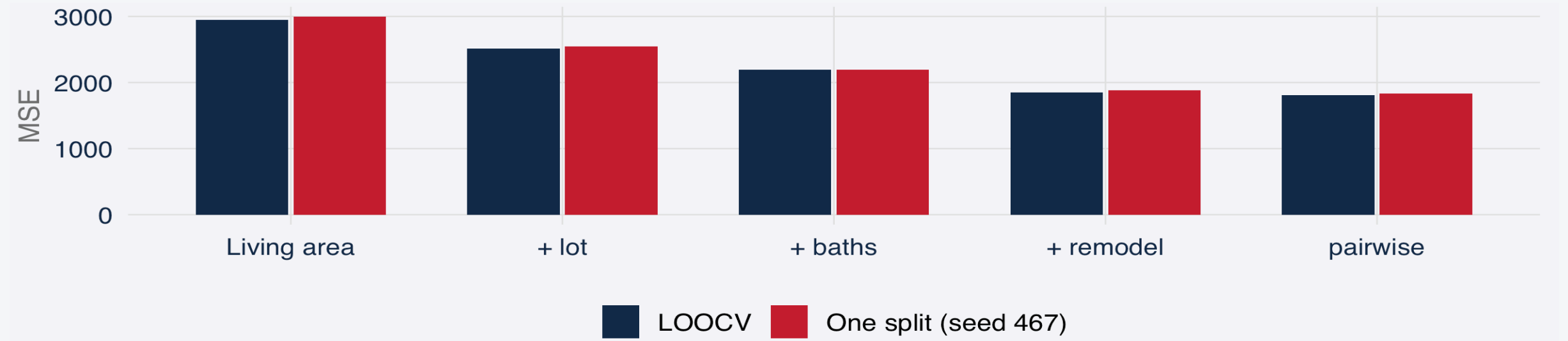
- For OLS. h_{ii} is the **leverage** of house i . One fit, not 5,802.

The OLS shortcut in R

```
1 m <- lm(TOTAL.VALUE ~ LIVING.AREA, data = houses)
2 h <- hatvalues(m)
3 cv_n <- mean(((houses$TOTAL.VALUE - fitted(m)) / (1 - h))^2)
```

- $CV_{(n)}$ for the line in living area: 2,946.1.
- Almost no bias; training size is $n - 1$; no random split. Slow if you really refit n times — the OLS shortcut fixes that. Can be a high-variance estimate of test error.

LOOCV versus one split



Both still fall through pairwise. On 5,802 houses extra nested columns still help.

k-fold on our data

We do not want 5,802 fits. If we cut the file into **10** blocks and hold out each block once, what number do we report — the last block's MSE, or something else?

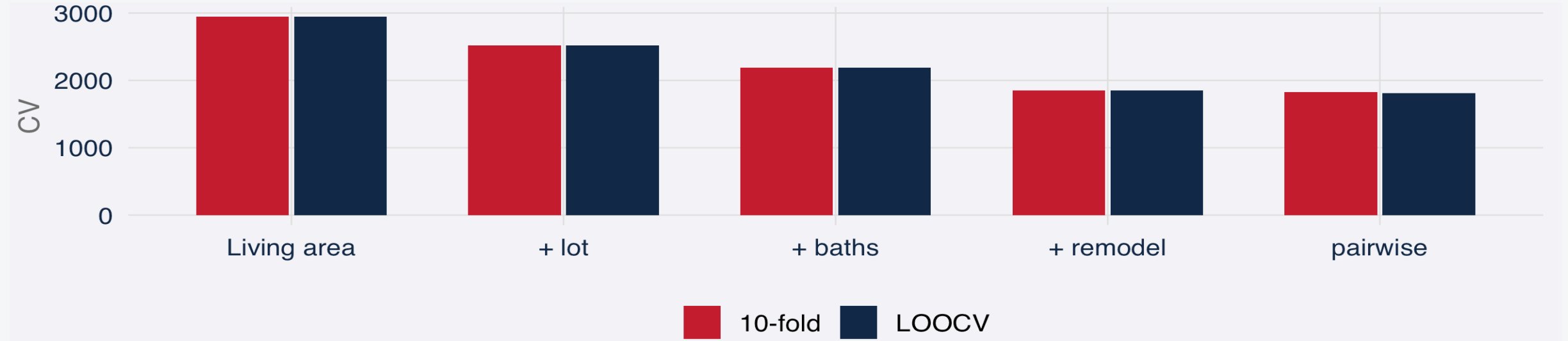
$$CV_{(k)} = \frac{1}{k} \sum_{\ell=1}^k \text{MSE}_{\ell}, \quad k = 10.$$

- LOOCV is this with $k = n$. The loop is the next slide.

A 10-fold loop in R

```
1 set.seed(467)
2 fold <- sample(rep(1:10, length.out = nrow(houses)))
3 cv_one <- function(form) {
4   mse <- numeric(10)
5   for (j in 1:10) {
6     m <- lm(form, data = houses[fold != j, ])
7     hold <- houses[fold == j, ]
8     mse[j] <- mean((hold$TOTAL.VALUE - predict(m, hold))^2)
9   }
10  mean(mse)
11 }
```

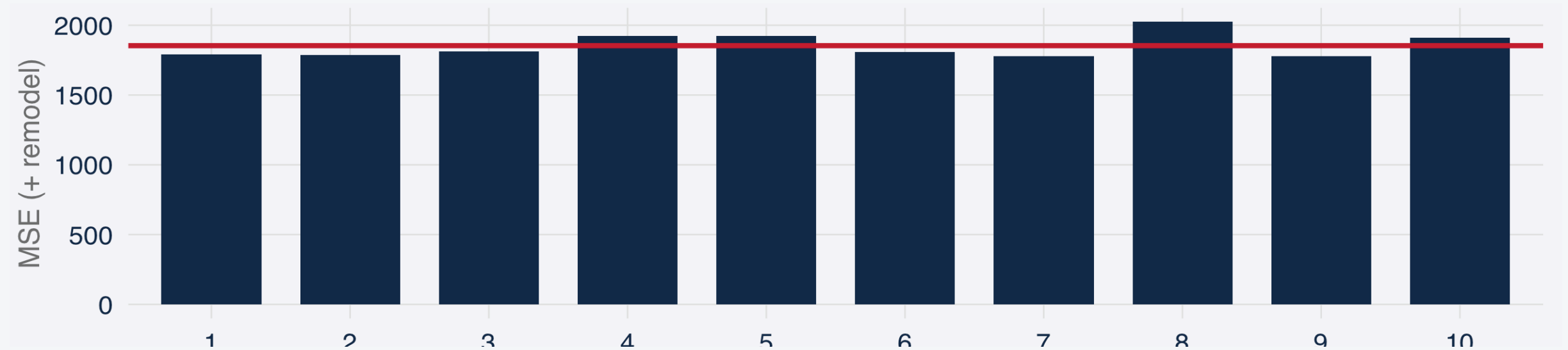
10-fold CV versus LOOCV



Same ranking. Cheaper than naive LOOCV. $k = 5$ or 10 is the usual compromise.

How the CV error is calculated

The loop just produced **ten** MSEs. Which of those ten do we take to the board — the smallest, the last, or something else?



Folds 1–10 for the + remodel model. Each bar is houses we did not fit that round. The red line is the mean — the number we report (1,853.6).

How we find the right complexity

On 80 houses, training MSE picked pairwise and the houses we did not fit picked + remodel. Using the CV numbers on the full fit which model do we take — and when is the training minimum the wrong pick?

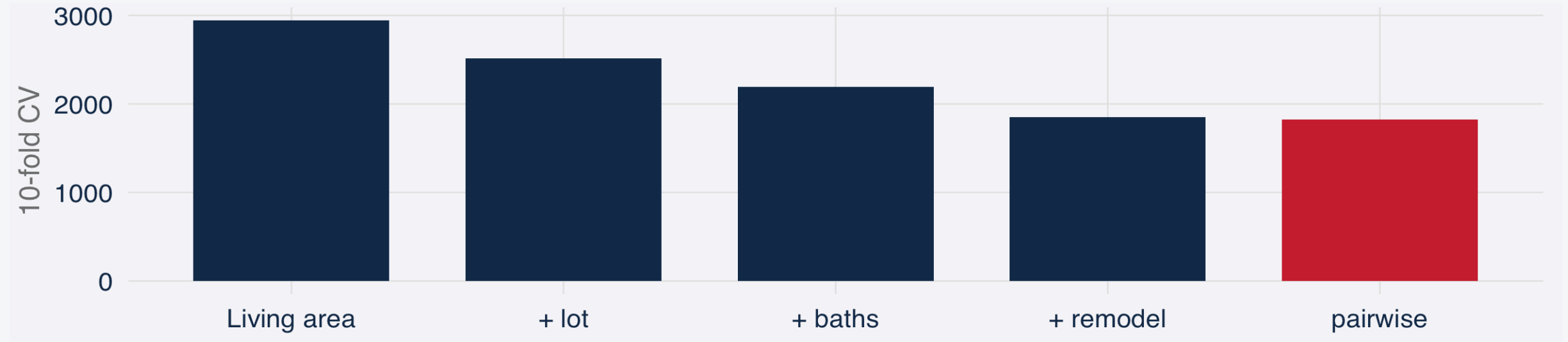
$$\hat{m} = \arg \min_m CV_{(k)}(m).$$

```
1 names(forms)[which.min(sapply(forms, cv_one))]
```

```
[1] "pairwise"
```

- Later in the course the same rule picks λ .

10-fold CV versus the model



On 5,802 houses CV picks pairwise — extra columns still help. On 80 houses the test pick is + remodel; training MSE on those 80 picked pairwise.

Before Wednesday, and Homework 1

Believe the error on houses you did not fit, and do not use information you would not have when the decision is made.

Wednesday

Honest evaluation in R, in person. Problem + starter before class. Bring a laptop.

Homework 1

Still last week's method. Do not put a train/test split or cross-validation in that notebook.

Today

The number you report is the test / CV number. TAX is a bill you would not have before you assess.

Appendix

Why the OLS shortcut equals leave-one-out

For OLS, $\hat{y} = Hy$ with $H = X(X^\top X)^{-1}X^\top$. The i th diagonal h_{ii} is the leverage of house i .

The prediction at i when house i is dropped satisfies

$$y_i - \hat{y}_i^{(-i)} = \frac{y_i - \hat{y}_i}{1 - h_{ii}}.$$

Square, average, and you have $CV_{(n)}$ from **one** fit. That is why we did not loop 5,802 times.

Posting a value is not adding a fireplace

The hour was about **scoring a prediction**. If the assessor posts $\hat{Y}$ for a house, that is not the same claim as “if we add a fireplace assessed value rises by $\hat{\beta}$.”

Prediction is not acting. Week 8 owns the long version.

Last week's intervals were still in-sample

A confidence interval for β_1 and a prediction interval for a new Y both use a model fit on the rows you already have.

Unless you split, both numbers are about **this** file. Today's test MSE is the number for a house you did not fit.